



Rajshahi University of Engineering & Technology (RUET)

Department of Electrical and Computer Engineering

Database Systems Sessional

Course Code: ECE 2216

Lab Report No. 04

SQL: Nested Subqueries, Aggregation, and Advanced Filtering

Submitted To,
Mst. Nafia Islam Shishir
Lecturer
Dept. of ECE, RUET

Submitted By,
Md. Yousuf
Roll: 2310023
Series: 2023
Dept. of ECE, RUET

Contents

1	Theory	3
2	Table Creation and Data Insertion	4
2.1	Table 1: Employee	4
3	Lab Tasks	5
3.1	Task 1: Display employees whose salary is greater than the average salary of all employees using a nested subquery.	5
3.2	Task 2: Display employees whose salary is greater than all employees of the HR department.	5
3.3	Task 3: Display employees who belong to a department where the department's average salary is greater than 50000.	6
3.4	Task 4: Display employees whose salary is greater than the average salary of their own department.	7
3.5	Task 5: Display the employees who earn the third-highest salary.	8
3.6	Task 6: Display the names of employees whose salary is higher than that of every employee in the EEE department but lower than the highest salary in the company.	9
4	Discussion	10

List of Figures

3.1	Output of Task 1	5
3.2	Output of Task 2	6
3.3	Output of Task 3	7
3.4	Output of Task 4	8
3.5	Output of Task 5	8
3.6	Output of Task 6	9

Chapter 1

Theory

Structured Query Language (SQL) is highly effective for managing relational databases, particularly when using advanced techniques such as subqueries and aggregations. Subqueries are queries nested inside another SQL statement. They can return a single value, a single row, a single column, or a complete table, allowing for dynamic comparisons.

In this lab, several advanced clauses and operators are used:

- **Nested Subqueries:** Used extensively to compare a column's value against an aggregated result (e.g., comparing an individual's salary against the company's average salary).
- **Correlated Subqueries:** A subquery that uses values from the outer query. It is evaluated once for each row processed by the outer query, enabling row-by-row comparisons within specific groups, like departments.
- **The ALL Operator:** Compares a value to every value in a list or results from a query. For instance, finding a salary greater than all salaries in a specific department.
- **GROUP BY and HAVING:** GROUP BY aggregates records into summary rows based on one or more columns, while HAVING filters these aggregated groups based on specified conditions.

Chapter 2

Table Creation and Data Insertion

A table named `Employee` was created for this lab to execute advanced filtering and subqueries.

2.1 Table 1: Employee

```
1 CREATE TABLE Employee (  
2     ID VARCHAR(10) PRIMARY KEY,  
3     Name VARCHAR(50),  
4     Gender VARCHAR(10),  
5     Department VARCHAR(10),  
6     Designation VARCHAR(20),  
7     Salary INT,  
8     Age INT,  
9     Email VARCHAR(50)  
10 );
```

Listing 2.1: Creating the Employee table

```
1 INSERT INTO Employee VALUES  
2 ('2310023', 'Yousuf Ahmed', 'Male', 'CSE', 'Lecturer', 52000, 23,  
   'yousuf@company.com'),  
3 ('2310024', 'Nabila Islam', 'Female', 'EEE', 'Engineer', 48000,  
   22, 'nabila@company.com'),  
4 ('2310025', 'Karim Hasan', 'Male', 'CSE', 'Programmer', 60000,  
   24, 'karim@company.com'),  
5 ('2310026', 'Sumaiya Akter', 'Female', 'HR', 'Executive', 42000,  
   21, 'sumaiya@company.com'),  
6 ('2310027', 'Rafi Khan', 'Male', 'EEE', 'Engineer', 55000, 23, '  
   rafi@company.com'),  
7 ('2310028', 'Anika Rahman', 'Female', 'CSE', 'Lecturer', 65000,  
   24, 'anika@company.com'),  
8 ('2310029', 'Hasan Ali', 'Male', 'HR', 'Manager', 70000, 27, '  
   hasan@company.com'),  
9 ('2310030', 'Sadiah Noor', 'Female', 'EEE', 'Technician', 39000,  
   22, 'sadia@company.com');
```

Listing 2.2: Inserting records into the Employee table

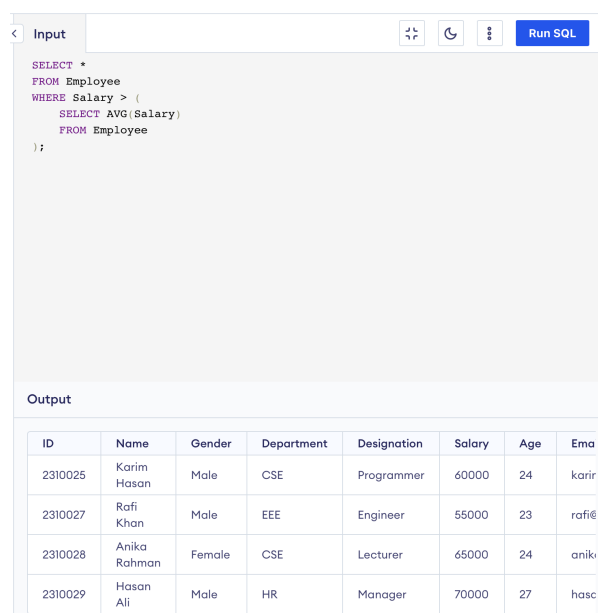
Chapter 3

Lab Tasks

3.1 Task 1: Display employees whose salary is greater than the average salary of all employees using a nested subquery.

```
1 SELECT *
2 FROM Employee
3 WHERE Salary > (
4     SELECT AVG(Salary)
5     FROM Employee
6 );
```

Listing 3.1: Query for Task 1



The screenshot shows a SQL query editor with the following query in the 'Input' tab:

```
SELECT *
FROM Employee
WHERE Salary > (
    SELECT AVG(Salary)
    FROM Employee
);
```

The 'Output' tab displays a table with 8 columns: ID, Name, Gender, Department, Designation, Salary, Age, and Email. The table contains 5 rows of data:

ID	Name	Gender	Department	Designation	Salary	Age	Email
2310025	Karim Hasan	Male	CSE	Programmer	60000	24	karir
2310027	Rafi Khan	Male	EEE	Engineer	55000	23	rafi@
2310028	Anika Rahman	Female	CSE	Lecturer	65000	24	aniki
2310029	Hasan Ali	Male	HR	Manager	70000	27	hasc

Figure 3.1: Output of Task 1

3.2 Task 2: Display employees whose salary is greater than all employees of the HR department.

```
1 SELECT *
2 FROM Employee
3 WHERE Salary > ALL (
4     SELECT Salary
```

```

5 FROM Employee
6 WHERE Department = 'HR'
7 );

```

Listing 3.2: Query for Task 2

The screenshot shows a SQL query editor with the following query:

```

SELECT *
FROM Employee
WHERE Salary > (
    SELECT Salary
    FROM Employee
    WHERE Department = 'HR'
);

```

The output is a table with 9 rows and 9 columns:

ID	Name	Gender	Department	Designation	Salary	Age	Ema
2310023	Yousuf Ahmed	Male	CSE	Lecturer	52000	23	yous
2310024	Nabila Islam	Female	EEE	Engineer	48000	22	nabi
2310025	Karim Hasan	Male	CSE	Programmer	60000	24	karir
2310027	Rafi Khan	Male	EEE	Engineer	55000	23	rafi
2310028	Anika Rahman	Female	CSE	Lecturer	65000	24	anik
2310029	Hasan Ali	Male	HR	Manager	70000	27	hasc

Figure 3.2: Output of Task 2

3.3 Task 3: Display employees who belong to a department where the department's average salary is greater than 50000.

```

1 SELECT *
2 FROM Employee
3 WHERE Department IN (
4     SELECT Department
5     FROM Employee
6     GROUP BY Department
7     HAVING AVG(Salary) > 50000
8 );

```

Listing 3.3: Query for Task 3

Input							
<pre> SELECT * FROM Employee WHERE Department IN (SELECT Department FROM Employee GROUP BY Department HAVING AVG(Salary) > 50000); </pre>							
Output							
ID	Name	Gender	Department	Designation	Salary	Age	Ema
2310023	Yousuf Ahmed	Male	CSE	Lecturer	52000	23	yous
2310025	Karim Hasan	Male	CSE	Programmer	60000	24	kari
2310026	Sumaiya Akter	Female	HR	Executive	42000	21	sum
2310028	Anika Rahman	Female	CSE	Lecturer	65000	24	anik
2310029	Hasan Ali	Male	HR	Manager	70000	27	hasc

Figure 3.3: Output of Task 3

3.4 Task 4: Display employees whose salary is greater than the average salary of their own department.

```

1 SELECT *
2 FROM Employee E1
3 WHERE Salary > (
4     SELECT AVG(Salary)
5     FROM Employee E2
6     WHERE E1.Department = E2.Department
7 );

```

Listing 3.4: Query for Task 4

Input							
<pre> SELECT * FROM Employee E1 WHERE Salary > (SELECT AVG(Salary) FROM Employee E2 WHERE E1.Department = E2.Department); </pre>							
Output							
ID	Name	Gender	Department	Designation	Salary	Age	Email
2310024	Nabila Islam	Female	EEE	Engineer	48000	22	nabi
2310025	Karim Hasan	Male	CSE	Programmer	60000	24	karim
2310027	Rafi Khan	Male	EEE	Engineer	55000	23	rafi
2310028	Anika Rahman	Female	CSE	Lecturer	65000	24	anik
2310029	Hasan Ali	Male	HR	Manager	70000	27	hasc

Figure 3.4: Output of Task 4

3.5 Task 5: Display the employees who earn the third-highest salary.

```

1 SELECT *
2 FROM Employee
3 ORDER BY Salary DESC
4 LIMIT 1 OFFSET 2;

```

Listing 3.5: Query for Task 5

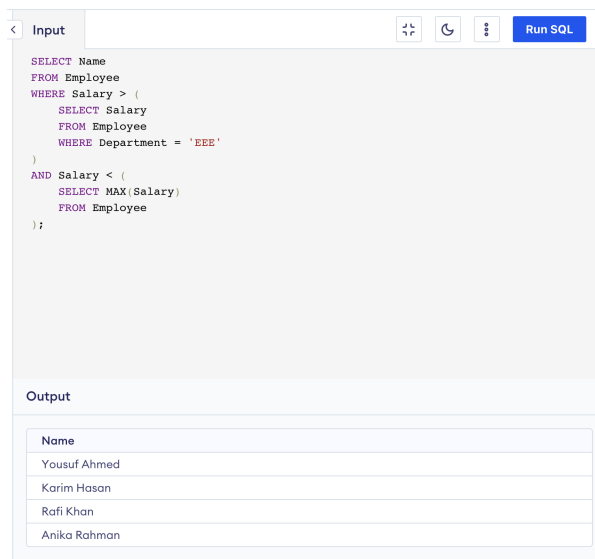
Input							
<pre> SELECT * FROM Employee ORDER BY Salary DESC LIMIT 1 OFFSET 2; </pre>							
Output							
ID	Name	Gender	Department	Designation	Salary	Age	Email
2310025	Karim Hasan	Male	CSE	Programmer	60000	24	karim

Figure 3.5: Output of Task 5

3.6 Task 6: Display the names of employees whose salary is higher than that of every employee in the EEE department but lower than the highest salary in the company.

```
1 SELECT Name
2 FROM Employee
3 WHERE Salary > ALL (
4     SELECT Salary
5     FROM Employee
6     WHERE Department = 'EEE'
7 )
8 AND Salary < (
9     SELECT MAX(Salary)
10    FROM Employee
11 );
```

Listing 3.6: Query for Task 6



The screenshot shows a SQL query editor with the following query in the 'Input' tab:

```
SELECT Name
FROM Employee
WHERE Salary > (
    SELECT Salary
    FROM Employee
    WHERE Department = 'EEE'
)
AND Salary < (
    SELECT MAX(Salary)
    FROM Employee
);
```

The 'Output' tab shows the results of the query in a table with the following data:

Name
Yousuf Ahmed
Karim Hasan
Rafi Khan
Anika Rahman

Figure 3.6: Output of Task 6

Chapter 4

Discussion

This lab provided a strong foundational understanding of executing complex conditional queries using SQL. Through nested subqueries, it became evident how relational databases can dynamically calculate variables—like an average salary—and instantly apply them as filters in the outer query.

Task 4 demonstrated the utility of correlated subqueries, showcasing how row-by-row comparisons against specific relational groups (e.g., departments) allow for deep contextual data retrieval without hardcoding limits. The **ALL** keyword was implemented efficiently to scale strict numerical boundaries across variable result subsets. Overall, these exercises highlighted the robust capabilities of SQL when handling layered, multi-condition logic within an organizational dataset.